PYTHON ACTIVITY-BASED LEARNING

Complete Documentation for 10 Selected Case Studies

Student Name	Saggurthi Yamini Durga
Roll Number	26000977
Branch / Section	CSM2
Subject	Python Programming

This document contains ten Python case studies with the required sections. Example rules and assumptions are stated for learning; confirm policy-specific requirements with your lecturer.

Important: Sample outputs are references only. Run each program and capture your own output screenshots before submission.

Contents

1. Examination Result and Grace-Mark Validation (Case Study 11)

2. Attendance Recovery Computation (Case Study 12)

3. ATM Transaction Sequence Simulation (Case Study 13)

4. Student Performance Trend Analysis (Case Study 17)

5. Traffic Violation Penalty Calculator (Case Study 21)

6. Password Policy Validation Engine (Case Study 29)

7. Text Compression Using Character Runs (Case Study 35)

8. Student Marks Matrix Analysis (Case Study 42)

9. Tuple-Based Product Catalogue Audit (Case Study 49)

10. Food Waste Monitoring System (Case Study 53)

Every case study includes: problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, sample output, and conclusion.

1. Examination Result and Grace-Mark Validation (Case Study 11)

1. Problem Statement

Develop a program to validate marks in three subjects, apply a limited grace-mark rule when eligible, and determine the final result.

2. Objectives

• Validate marks from 0 to 100.

• Apply grace marks only under the stated example policy.

• Calculate total, percentage, and final result.

3. Assumptions

• Three subjects, each out of 100; pass mark is 35.

• At most 2 grace marks may be given in total if exactly one subject is short by 1 or 2 marks.

• Confirm this example policy with the lecturer.

4. Algorithm

1. Input three marks.

2. Reject values outside 0–100.

3. If exactly one subject is short by 1 or 2 marks, add the required grace marks.

4. Calculate total and percentage.

5. Display PASS or FAIL.

5. Flowchart

START → Input marks → Validate → Check grace eligibility → Calculate total/percentage → Display result → STOP

6. Pseudocode

```
INPUT three marks
IF any mark outside 0..100: PRINT invalid
ELSE apply grace only if exactly one subject is short by 1 or 2
Calculate total and percentage
PRINT PASS if every adjusted mark >= 35, otherwise FAIL
```

7. Python Source Code

```
marks = [int(input(f"Marks for subject {i}: ")) for i in range(1, 4)]
if any(m < 0 or m > 100 for m in marks):
    print("Invalid marks")
else:
    weak = [i for i, m in enumerate(marks) if m < 35]
    grace = 0
    if len(weak) == 1 and 1 <= 35 - marks[weak[0]] <= 2:
        grace = 35 - marks[weak[0]]
        marks[weak[0]] = 35
    total = sum(marks)
    print("Final marks:", marks)
    print("Grace marks used:", grace)
    print("Total:", total, "/ 300")
    print(f"Percentage: {total / 3:.2f}%")
    print("Result:", "PASS" if all(m >= 35 for m in marks) else "FAIL")
```

8. Test Cases

Input / Test	Expected result
70,80,90	PASS; total 240; 80.00%
34,60,70	1 grace mark; PASS
33,60,70	2 grace marks; PASS
30,60,70	FAIL
110,60,70	Invalid marks

9. Sample Output (run the program to capture your own screenshot)

```
Marks for subject 1: 70
Marks for subject 2: 80
```

```
Marks for subject 3: 90
Final marks: [70, 80, 90]
Grace marks used: 0
Total: 240 / 300
Percentage: 80.00%
Result: PASS
```

10. Conclusion

Validates marks and applies the stated example grace policy before calculating the final result.

2. Attendance Recovery Computation (Case Study 12)

1. Problem Statement

Calculate current attendance and estimate the minimum number of future classes a student must attend to reach a target percentage.

2. Objectives

• Calculate current attendance percentage.

• Find the minimum additional classes needed.

• Validate attendance counts.

3. Assumptions

• Attended classes cannot exceed classes held.

• Target percentage is between 0 and 100.

• Every future class is attended.

4. Algorithm

1. Input held, attended, and target percentage.

2. Validate inputs.

3. Calculate current percentage.

4. Increase future attended and held counts until the target is met.

5. Display current percentage and required classes.

5. Flowchart

START → Input data → Validate → Calculate current percentage → Simulate future attendance → Display → STOP

6. Pseudocode

```
INPUT held, attended, target
VALIDATE counts and target
current = attended / held * 100 (if held > 0)
future = 0
WHILE target not reached after future attended classes: future += 1
PRINT current and future
```

7. Python Source Code

```
held = int(input("Classes held: "))
attended = int(input("Classes attended: "))
target = float(input("Target percentage: "))
if held < 0 or attended < 0 or attended > held or not 0 <= target <= 100:
    print("Invalid attendance data")
else:
    current = attended / held * 100 if held else 0
    future = 0
    while target > 0 and (attended + future) / (held + future) * 100 < target:
        future += 1
    print(f"Current attendance: {current:.2f}%")
    print("Additional classes to attend:", future)
```

8. Test Cases

Input / Test	Expected result
Held 40, attended 28, target 75	70.00%; 8 more classes
Held 20, attended 18, target 75	90.00%; 0 more classes
Held 10, attended 12	Invalid attendance data

9. Sample Output (run the program to capture your own screenshot)

```
Classes held: 40
Classes attended: 28
Target percentage: 75
Current attendance: 70.00%
Additional classes to attend: 8
```

10. Conclusion

Estimates the number of consecutive classes needed to reach a required attendance percentage.

3. ATM Transaction Sequence Simulation (Case Study 13)

1. Problem Statement

Simulate deposits and withdrawals, update the account balance, reject invalid transactions, and summarize results.

2. Objectives

• Process deposits and withdrawals.

• Prevent invalid amounts and overdrafts.

• Display final balance and transaction counts.

3. Assumptions

• Opening balance is non-negative.

• Use D for deposit and W for withdrawal.

• Transaction amounts must be positive.

4. Algorithm

1. Input opening balance and transaction count.

2. For each transaction, input type and amount.

3. Accept valid deposits and affordable withdrawals.

4. Reject other transactions.

5. Display balance and counts.

5. Flowchart

START → Input balance/count → Process each transaction → Validate/update or reject → Display summary → STOP

6. Pseudocode

```
INPUT balance, n
FOR n transactions: INPUT type and amount
IF amount <= 0: reject
ELSE IF type D: add amount
ELSE IF type W and amount <= balance: subtract amount
ELSE reject
PRINT balance and counts
```

7. Python Source Code

```
balance = float(input("Opening balance: "))
n = int(input("Number of transactions: "))
ok = rejected = 0
if balance < 0 or n < 0:
    print("Invalid opening balance or count")
else:
    for _ in range(n):
        kind = input("Type (D/W): ").strip().upper()
        amount = float(input("Amount: "))
        if amount <= 0:
            print("Rejected: amount must be positive")
            rejected += 1
        elif kind == "D":
            balance += amount
            ok += 1
        elif kind == "W" and amount <= balance:
            balance -= amount
            ok += 1
        else:
            print("Rejected: invalid type or insufficient balance")
            rejected += 1
    print(f"Final balance: {balance:.2f}")
    print("Successful:", ok, "Rejected:", rejected)
```

8. Test Cases

Input / Test	Expected result
Opening 500; D 200; W 100	Final balance 600.00
Opening 100; W 150	Withdrawal rejected; balance 100.00
Deposit -10	Rejected

9. Sample Output (run the program to capture your own screenshot)

```
Opening balance: 500
Number of transactions: 2
Type (D/W): D
Amount: 200
Type (D/W): W
Amount: 100
Final balance: 600.00
Successful: 2 Rejected: 0
```

10. Conclusion

Demonstrates safe balance updates and transaction validation in an educational simulation.

4. Student Performance Trend Analysis (Case Study 17)

1. Problem Statement

Compare marks across two exams, calculate changes, identify the weakest subject, and classify overall performance.

2. Objectives

• Calculate subject-wise score changes.

• Find the lowest-scoring subject.

• Classify the average score.

3. Assumptions

• Three subjects are scored out of 100 in two exams.

• Average >=75 is High, >=50 is Moderate, otherwise Needs Improvement.

4. Algorithm

1. Input marks for both exams.

2. Validate marks.

3. Calculate score changes and second-exam average.

4. Identify weakest subject and category.

5. Display report.

5. Flowchart

START → Input two sets of marks → Validate → Calculate changes/average → Find weakest subject → Classify → STOP

6. Pseudocode

```
INPUT first and second marks for each subject
IF any mark outside 0..100: error
ELSE calculate change = second - first
Calculate average, weakest subject, and category
PRINT report
```

7. Python Source Code

```
subjects = ["Python", "Mathematics", "Communication"]
first = [int(input(f"{s} first exam: ")) for s in subjects]
second = [int(input(f"{s} second exam: ")) for s in subjects]
if any(m < 0 or m > 100 for m in first + second):
    print("Invalid marks")
else:
    for i, s in enumerate(subjects):
        print(f"{s} change: {second[i] - first[i]:+d}")
    avg = sum(second) / 3
    print(f"Second exam average: {avg:.2f}")
    print("Weakest subject:", subjects[second.index(min(second))])
    print("Category:", "High" if avg >= 75 else "Moderate" if avg >= 50 else "Needs Improvement")
```

8. Test Cases

Input / Test	Expected result
First 60,70,80; second 70,65,90	Changes +10,-5,+10; average 75; High
Second marks 40,45,50	Average 45; Needs Improvement
Any mark 105	Invalid marks

9. Sample Output (run the program to capture your own screenshot)

```
Python first exam: 60
Mathematics first exam: 70
Communication first exam: 80
Python second exam: 70
Mathematics second exam: 65
Communication second exam: 90
Python change: +10
Mathematics change: -5
Communication change: +10
Second exam average: 75.00
Weakest subject: Mathematics
Category: High
```

10. Conclusion

Summarizes score improvements, the weakest subject, and overall performance category.

5. Traffic Violation Penalty Calculator (Case Study 21)

1. Problem Statement

Calculate a hypothetical penalty based on offence type and repeat status using example classroom rates.

2. Objectives

• Select a fine from a rule table.

• Double the fine for a repeat offence.

• Display the calculated penalty.

3. Assumptions

• Example fines: speeding ■1000, signal violation ■2000, no helmet ■500.

• Repeat offences double the example fine.

• These are educational examples, not official current legal rates.

4. Algorithm

1. Input offence and repeat status.

2. Validate both values.

3. Look up base fine.

4. Double it if repeated.

5. Display the example penalty.

5. Flowchart

START → Input offence/repeat status → Validate → Look up fine → Apply repeat multiplier → Display → STOP

6. Pseudocode

```
INPUT offence, repeat status
IF offence not in table: error
ELSE fine = base fine
IF repeat: fine = fine * 2
PRINT example fine
```

7. Python Source Code

```
fines = {"speeding": 1000, "signal": 2000, "no helmet": 500}
offence = input("Offence (speeding/signal/no helmet): ").strip().lower()
repeat = input("Repeat offence? (yes/no): ").strip().lower()
if offence not in fines or repeat not in ("yes", "no"):
    print("Invalid input")
else:
    fine = fines[offence] * (2 if repeat == "yes" else 1)
    print(f"Example penalty: Rs. {fine}")
    print("Educational example only; check current official rules.")
```

8. Test Cases

Input / Test	Expected result
speeding, no	■1000
no helmet, yes	■1000
unknown, no	Invalid input

9. Sample Output (run the program to capture your own screenshot)

```
Offence: no helmet
Repeat offence? (yes/no): yes
Example penalty: Rs. 1000
Educational example only; check current official rules.
```

10. Conclusion

Shows conditional fine calculation using hypothetical rates; official fines may differ.

6. Password Policy Validation Engine (Case Study 29)

1. Problem Statement

Check a sample password against length, character-type, and prohibited-substring requirements.

2. Objectives

• Check minimum length.

• Check uppercase, lowercase, digit, and special character.

• Report all failed rules.

3. Assumptions

• Minimum length is 8.

• Require uppercase, lowercase, digit, and one of !@#$%&*.

• The substring 'password' is prohibited ignoring case.

4. Algorithm

1. Input a sample password.

2. Check every policy rule.

3. Collect and display failures.

4. Display VALID if there are no failures.

5. Flowchart

START → Input password → Check rules → Collect errors → Display VALID or errors → STOP

6. Pseudocode

```
INPUT password
CHECK length, uppercase, lowercase, digit, special character
CHECK prohibited substring ignoring case
IF no errors: PRINT VALID
ELSE PRINT errors
```

7. Python Source Code

```
password = input("Enter a sample password: ")
errors = []
if len(password) < 8: errors.append("Use at least 8 characters.")
if not any(c.isupper() for c in password): errors.append("Add uppercase letter.")
if not any(c.islower() for c in password): errors.append("Add lowercase letter.")
if not any(c.isdigit() for c in password): errors.append("Add a digit.")
if not any(c in "!@#$%&*" for c in password): errors.append("Add a special character.")
if "password" in password.lower(): errors.append("Do not use the word password.")
if errors:
    print("Password: INVALID")
    for e in errors: print("-", e)
else:
    print("Password: VALID")
```

8. Test Cases

Input / Test	Expected result
Hello@123	VALID
hello123	Uppercase and special character missing
Password@123	Prohibited substring

9. Sample Output (run the program to capture your own screenshot)

```
Enter a sample password: Hello@123
Password: VALID
```

10. Conclusion

Demonstrates string validation. Use sample passwords only, never real personal passwords.

7. Text Compression Using Character Runs (Case Study 35)

1. Problem Statement

Encode consecutive repeated characters as a character followed by its run length, then compare encoded and original lengths.

2. Objectives

• Count consecutive character runs.

• Build the encoded string.

• Recommend encoding only when it is shorter.

3. Assumptions

• Each run is represented as character plus count, including count 1.

• Empty input is handled separately.

4. Algorithm

1. Input text.

2. Track current character and count.

3. Append each completed run.

4. Append the last run.

5. Compare lengths and display preferred representation.

5. Flowchart

START → Input text → Count runs → Build encoded string → Compare lengths → Display result → STOP

6. Pseudocode

```
INPUT text
IF empty: report empty
ELSE scan runs and append character + count
PRINT encoded text and lengths
IF encoded shorter: recommend encoded; ELSE recommend original
```

7. Python Source Code

```
text = input("Enter text to compress: ")
if not text:
    print("Input is empty")
else:
    encoded = ""
    current = text[0]
    count = 1
    for ch in text[1:]:
        if ch == current:
            count += 1
        else:
            encoded += current + str(count)
            current, count = ch, 1
    encoded += current + str(count)
    print("Original:", text)
    print("Encoded:", encoded)
    print("Original length:", len(text))
    print("Encoded length:", len(encoded))
    print("Preferred:", "encoded" if len(encoded) < len(text) else "original")
```

8. Test Cases

Input / Test	Expected result
aaabbc	a3b2c1; not shorter
aaaaabbbb	a5b4; shorter
abc	a1b1c1; original preferred

9. Sample Output (run the program to capture your own screenshot)

```
Enter text to compress: aaaaabbbb
Original: aaaaabbbb
Encoded: a5b4
Original length: 9
Encoded length: 4
Preferred: encoded
```

10. Conclusion

Demonstrates run-length encoding and checks whether it reduces the text length.

8. Student Marks Matrix Analysis (Case Study 42)

1. Problem Statement

Use a nested list to analyse marks for several students and subjects, calculate totals and averages, and find the top student.

2. Objectives

• Store marks in a nested list.

• Calculate student totals and subject averages.

• Identify the top student and low-average subjects.

3. Assumptions

• Three students and three subjects.

• Marks are integers from 0 to 100.

• Flag a subject if its average is below 50.

4. Algorithm

1. Input a 3×3 marks matrix.

2. Validate all marks.

3. Calculate row totals and column averages.

4. Find the top student and subjects averaging below 50.

5. Display report.

5. Flowchart

START → Input marks matrix → Validate → Calculate totals/averages → Find top student/low subjects → Display → STOP

6. Pseudocode

```
INPUT marks for 3 students and 3 subjects
IF any mark outside 0..100: error
ELSE calculate row totals and column averages
Find highest total and averages below 50
PRINT report
```

7. Python Source Code

```
students = ["Asha", "Balu", "Charan"]
subjects = ["Python", "Maths", "English"]
marks = []
for student in students:
    row = [int(input(f"{student} - {subject}: ")) for subject in subjects]
    marks.append(row)
if any(m < 0 or m > 100 for row in marks for m in row):
    print("Invalid marks")
else:
    totals = [sum(row) for row in marks]
    avgs = [sum(marks[i][j] for i in range(3)) / 3 for j in range(3)]
    print("Student totals:", dict(zip(students, totals)))
    print("Subject averages:", dict(zip(subjects, [round(a, 2) for a in avgs])))
    print("Top student:", students[totals.index(max(totals))])
    print("Subjects averaging below 50:", [subjects[j] for j, a in enumerate(avgs) if a < 50])
```

8. Test Cases

Input / Test	Expected result
80,70,90; 60,50,70; 40,45,50	Totals 240,180,135; Asha top
Any mark -1 or 101	Invalid marks

9. Sample Output (run the program to capture your own screenshot)

```
Student totals: {'Asha': 240, 'Balu': 180, 'Charan': 135}
Subject averages: {'Python': 60.0, 'Maths': 55.0, 'English': 70.0}
Top student: Asha
Subjects averaging below 50: []
```

10. Conclusion

Demonstrates nested lists and row/column analysis of student marks.

9. Tuple-Based Product Catalogue Audit (Case Study 49)

1. Problem Statement

Use tuples to store product ID, price, and quantity; validate records, calculate inventory value, and flag low stock.

2. Objectives

• Represent product records as tuples.

• Calculate valid inventory value.

• Find products requiring replenishment.

3. Assumptions

• Record format is (product ID, price, quantity).

• Price and quantity must be non-negative.

• Stock below 5 needs replenishment.

4. Algorithm

1. Read product tuples.

2. Reject records with negative price or quantity.

3. Add valid price × quantity to inventory value.

4. Flag valid products with quantity below 5.

5. Display summary.

5. Flowchart

START → Read tuples → Validate → Calculate value → Flag low stock → Display audit → STOP

6. Pseudocode

```
FOR each product (id, price, quantity)
IF price < 0 or quantity < 0: report invalid
ELSE add price * quantity to inventory value
IF quantity < 5: flag for replenishment
PRINT summary
```

7. Python Source Code

```
products = [("P101", 25.50, 12), ("P102", 80.00, 3),
            ("P103", 15.00, 0), ("P104", -5.00, 4)]
value = 0.0
replenish, invalid = [], []
for pid, price, qty in products:
    if price < 0 or qty < 0:
        invalid.append(pid)
        continue
    value += price * qty
    if qty < 5:
        replenish.append(pid)
print(f"Valid inventory value: Rs. {value:.2f}")
print("Products needing replenishment:", replenish)
print("Invalid records:", invalid)
```

8. Test Cases

Input / Test	Expected result
P101, 25.50, 12	Value ■306.00
P102, 80, 3	Needs replenishment
P104, -5, 4	Invalid record

9. Sample Output (run the program to capture your own screenshot)

```
Valid inventory value: Rs. 426.00
Products needing replenishment: ['P102', 'P103']
Invalid records: ['P104']
```

10. Conclusion

Uses tuples to calculate inventory value and identify low-stock or invalid products.

10. Food Waste Monitoring System (Case Study 53)

1. Problem Statement

Compare meals prepared and served across days, calculate leftovers and wastage percentages, and flag high-waste days.

2. Objectives

• Calculate leftovers and waste percentage.

• Compare waste against a threshold.

• Identify the highest-waste day.

3. Assumptions

• Prepared meals must be greater than zero.

• Served meals range from zero to prepared meals.

• Flag waste above 10%.

4. Algorithm

1. Input prepared and served meals for each day.

2. Validate counts.

3. Calculate leftovers and waste percentage.

4. Flag days above 10%.

5. Display highest-waste day.

5. Flowchart

START → Input daily meals → Validate → Calculate leftovers and waste % → Compare with 10% → Display daily summary → STOP

6. Pseudocode

```
FOR each day: INPUT prepared, served
IF prepared <= 0 or served < 0 or served > prepared: invalid
ELSE leftovers = prepared - served
waste% = leftovers / prepared * 100
Flag if waste% > 10
PRINT daily report and highest-waste day
```

7. Python Source Code

```
days = ["Monday", "Tuesday", "Wednesday"]
records = []
for day in days:
    prepared = int(input(f"{day} meals prepared: "))
    served = int(input(f"{day} meals served: "))
    if prepared <= 0 or served < 0 or served > prepared:
        print(day, ": invalid input")
    else:
        leftovers = prepared - served
        waste = leftovers / prepared * 100
        records.append((day, leftovers, waste))
        status = "Above limit" if waste > 10 else "Within limit"
        print(f"{day}: leftovers={leftovers}, waste={waste:.2f}% ({status})")
if records:
    print("Highest wastage day:", max(records, key=lambda r: r[2])[0])
```

8. Test Cases

Input / Test	Expected result
Prepared 100, served 90	10 leftovers; 10%; within limit
Prepared 100, served 80	20 leftovers; 20%; above limit
Prepared 0, served 0	Invalid input

9. Sample Output (run the program to capture your own screenshot)

```
Monday meals prepared: 100
Monday meals served: 90
Tuesday meals prepared: 100
Tuesday meals served: 80
Wednesday meals prepared: 50
```

```
Wednesday meals served: 48
Monday: leftovers=10, waste=10.00% (Within limit)
Tuesday: leftovers=20, waste=20.00% (Above limit)
Wednesday: leftovers=2, waste=4.00% (Within limit)
Highest wastage day: Tuesday
```

10. Conclusion

Calculates food leftovers and waste percentages, flags days exceeding the threshold, and identifies the highest-waste day.